

Inteligencia Artificial

Centro Universitario de Ciencias Exactas e Ingenierías



Proyecto final

Ingeniería en Computación

Profesor: Luis Ángel Beltrán Carrillo

Integrantes:

Vanessa Araceli Vázquez Mojica

Mariana Gonzales Velázquez

Braulio Yael Carranza Zamora

Objetivo:

Comprender el funcionamiento de las técnicas de optimización metaheurísticas, mediante su implementación en problemas matemáticos y problemas reales.

Desarrollo e Implementación:

Esta práctica se divide en varias etapas, en las que el alumno será capaz de recopilar información e implementar soluciones a problemas matemáticos y problemas reales. El alumno trabajara en equipos de máximo tres integrantes.

• Algoritmos Metaheurísticos

Los algoritmos matehurísticos son algoritmos aproximados de optimización y búsqueda de propósito general, son procedimientos iterativos que guían una heurística subordinada combinando de forma inteligente distintos conceptos para explorar y explotar adecuadamente el espacio de búsqueda.

Los factores que hacen interesante su uso, es cuando no hay un método exacto de resolución, o este requiere mucho tiempo de calculo y memoria; cuando no se necesita una solución óptima, basta con una buena calidad.

El equilibrio entre intensificación y diversificación es necesario para identificar rápidamente regiones del espacio con soluciones de buena calidad y no consumir mucho tiempo en regiones del espacio no prometedoras o ya exploradas.

Su clasificación puede ser: Basadas en métodos constructivos (GRASP, optimización basada en colonias de hormigas), Basadas en trayectorias (búsqueda local, Enfriamiento simulado, búsqueda Tabú, BL iterativa, etc.), Basadas en poblaciones (Algoritmos genéticos, Scatter Search, Algoritmos Meméticos, PSO, etc.).

Las heurísticas constructivas son más rápidas pero dan soluciones de peor calidad que la búsqueda basada en trayectorias. En los métodos constructivos, el espacio es de soluciones parciales, mientras que en la búsqueda por trayectorias y poblaciones es de soluciones completas (candidatas). En este ultimo caso, el espacio de búsqueda suele ser de un tamaño exponencial con respecto al tamaño del problema.

Su aplicación puede ser en la optimización combinatoria, optimización en ingeniería, Modelado e identificación de sistemas, planificación y control, Aprendizaje y minería de datos, vida artificial y Bioinformática.

• Particle Swarm Optimization (PSO) (Optimización por enjambre de partículas)

Es una técnica de optimización/búsqueda, normalmente PSO se usa en espacios de búsquedas con muchas dimensiones. Este método fue descrito alrededor de 1995 por Kennedy y Eberhart, y se inspira en el comportamiento de los enjambres de insectos en la naturaleza. Podemos pensar en un enjambre de abejas, ya que

éstas a la hora de recolectar polen buscan la región del espacio en la que existe más densidad de flores, porque la probabilidad de que haya polen es mayor.

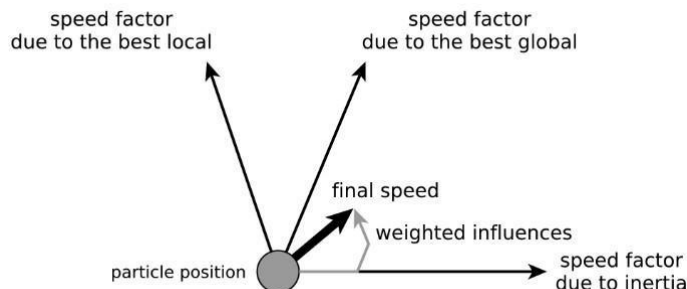
Formalmente hablando, se supone que tenemos una función desconocida, $f(x_1, \dots, x_n)$, que podemos evaluar en los puntos que queramos pero a modo de caja negra, por lo que no podemos conocer su expresión. El objetivo es el habitual en optimización, encontrar valores de $x_1, \dots, x_n$ para los que la función f sea máxima (o mínima, o bien verifica alguna relación extremal respecto a alguna otra función). Como ya hemos visto en otras situaciones similares, a f se le suele llamar **función de fitness**, ya que va a determinar cómo de buena es la posición actual para cada partícula.

Una primera aproximación para resolver este problema de calcular valores extremales de una función podría ser la selección aleatoria de valores de sus variables $x_1, \dots, x_n$, y almacenar el mayor de los resultados encontrados, lo que se conoce como una **búsqueda aleatoria**.

Funcionamiento de PSO

Cada partícula (individuo) tiene una **posición**, $p^{\rightarrow}$ (que en 2 dimensiones vendrá determinado por un vector de la forma (p_x, p_y)), en el espacio de búsqueda y una **velocidad**, $v^{\rightarrow}$ (que en 2 dimensiones vendrá determinado por un vector de la forma (v_x, v_y)), que determina su movimiento a través del espacio. Además, como partículas de un mundo real físico, tienen una cantidad de **inercia**, w , que los mantiene en la misma dirección en la que se movían, así como una aceleración (cambio de velocidad), que depende principalmente de dos características:

1. Cada partícula es atraída hacia la mejor localización que ella, personalmente, ha encontrado en su historia (p_{besti}).
2. Cada partícula es atraída hacia la mejor localización que ha sido encontrada por el conjunto de partículas en el espacio de búsqueda (p_{gbest}).



El algoritmo PSO se esboza a grandes rasgos en el siguiente código [4]:

```
Asignar posiciones y velocidades aleatorias iniciales a las partículas
```

```
Repetir
```

```
    Cada partícula:
```

```
        Actualizar su velocidad considerando:
```

Inercia de la partícula (la hace seguir con la misma velocidad)
Atracción al mejor personal
Atracción al mejor global
Actualizar la posición de la partícula
Calcular el valor de fitness en la nueva posición
Actualizar su mejor personal
Actualizar el mejor global del sistema
Devolver el mejor global

Lo interesante de este modelo es que cada partícula solo hace operaciones vectoriales básicas (para actualizar las velocidades y posiciones basta realizar algunas sumas de vectores) y, como la experiencia muestra que hacen falta pocas partículas y el número de iteraciones es bajo, la complejidad del proceso se centra sobre todo en la evaluación del fitness de cada partícula en cada iteración, algo que no puede evitarse y que depende exclusivamente del problema específico que se quiera optimizar.

En consecuencia, este método de optimización es especialmente adecuado cuando el coste de la función de fitness es muy elevado,

- **Genetic Algorithm (GA) (Algoritmo Genético). Usar la versión para números reales.**

Es una técnica de optimización inspirada en los principios de la evolución biológica. Su funcionamiento se basa en la selección natural, el cruce genérico y la mutación. En la variante con codificación real, cada solución candidata se representa como un vector de números reales.

El proceso inicia con la generación aleatoria de una población inicial dentro de los rangos permitidos para cada variable. Cada individuo es evaluado mediante una función de aptitud (fitness), que determina la calidad de la solución. A partir de estos valores se seleccionan los padres que participaran en la reproducción; un método común es la selección por torneo, donde se elige el mejor individuo dentro de un pequeño grupo seleccionado al azar.

La reproducción se realiza combinando las características de dos padres para generar uno o más descendientes.

En la codificación real, los operadores de cruce más utilizados son:

a) Cruce BLX- α (Blend Crossover)

Genera valores dentro de un intervalo extendido alrededor de los valores de los padres:

$$c_i \sim U[\min(p_{1i}, p_{2i}) - \alpha \cdot d_i, \max(p_{1i}, p_{2i}) + \alpha \cdot d_i]$$

donde $d_i = |p_{1i} - p_{2i}|$.

b) Cruce SBX (Simulated Binary Crossover)

Imita matemáticamente el comportamiento del cruce binario, manteniendo una buena exploración del espacio de búsqueda.

La **reproducción** se realiza mediante operadores de cruce diseñados para valores reales. Entre los más utilizados se encuentran el **BLX- α** , que genera descendientes dentro de un intervalo extendido entre ambos padres, y el **SBX (Simulated Binary Crossover)**, que simula el comportamiento del cruce binario sobre valores continuos. Posteriormente, los nuevos individuos pueden experimentar algún tipo de **mutación**, generalmente gaussiana, que consiste en agregar una pequeña perturbación aleatoria a uno o más genes. Esto permite mantener diversidad genética y evitar la convergencia prematura.

Después de generar los nuevos individuos, el algoritmo decide cuáles formarán parte de la siguiente generación. Dos estrategias frecuentes son:

- **Reemplazo generacional:** toda la población se sustituye por los descendientes.
- **Elitismo:** el mejor individuo de la generación se conserva automáticamente.

El elitismo garantiza que la mejor solución encontrada hasta el momento no se pierda.

En conjunto, los AG con codificación real permiten realizar una búsqueda efectiva en problemas de optimización continua, equilibrando exploración y explotación mediante operadores adaptados al manejo de variables reales.

• Differential Evolution (DE) (Evolución Diferencial)

La **Evolución Diferencial (Differential Evolution, DE)** es un método de optimización poblacional orientado a problemas continuos y multidimensionales. El algoritmo opera sobre una población de vectores reales y genera nuevas soluciones mediante la combinación lineal de individuos existentes. Para cada individuo objetivo x , se seleccionan aleatoriamente tres individuos distintos de la población, denotados como a , b y c . A partir de ellos se construye un **vector mutante** mediante la expresión: **$v = a + F(b - c)$**

donde F es el **factor de escala**, un parámetro en el rango $[0,1]$ que controla la magnitud de la perturbación diferencial. Posteriormente, el vector mutante v se combina con el individuo objetivo x mediante un proceso de **recombinación**, generalmente aplicado componente por componente con una probabilidad denominada **CR** (*crossover rate*), dando lugar a un vector hijo u .

Una vez generado el hijo, se realiza una **selección por reemplazo directo**: si el vector hijo u obtiene un valor de función objetivo superior al del padre x , entonces u pasa a formar parte de la siguiente generación; en caso contrario, se conserva x . Este mecanismo garantiza que la población no empeore y al mismo tiempo favorece la convergencia hacia soluciones de mayor calidad.

El proceso de mutación diferencial, recombinación y selección se repite de manera iterativa hasta satisfacer un criterio de terminación, como un número máximo de generaciones o la estabilidad del valor óptimo. La Evolución Diferencial se caracteriza por su simplicidad, robustez y excelente desempeño en problemas continuos, especialmente en aquellos donde el paisaje de la función objetivo presenta múltiples óptimos locales.

Lampinen y Zelinka propusieron una extensión del algoritmo DE para la optimización mixta-discreta, incluyendo los métodos necesarios para el manejo de parámetros discretos y enteros. Desarrollaron un método eficaz para el manejo de restricciones, que no requiere que el usuario defina parámetros de búsqueda adicionales, salvo proporcionar las funciones de restricción al programar la función de evaluación.

La modificación reside en la condición de selección. El miembro de la siguiente generación se selecciona según el siguiente criterio:

1. Se seleccionará el vector de prueba:
 - cuando ambas soluciones comparadas son factibles y el vector de prueba satisface todas las restricciones y proporciona un valor de función objetivo menor o igual que el del vector actual.
 - cuando el vector de prueba es factible mientras que el vector actual no lo es.
 - si el vector de prueba es inviable, pero proporciona una violación de restricción menor o igual para todas las funciones de restricción.
2. Si ambas soluciones comparadas son factibles, se selecciona la que tenga el valor de función objetivo más bajo.
3. Si ambas soluciones comparadas son inviables, entonces el vector de prueba se considera menos inviable y mejor que el vector actual, si no viola ninguna de las restricciones más que el vector actual y si viola al menos una de las restricciones menos.

La Evolución Diferencial (ED) es un método poblacional para problemas de optimización global. La ED es estocástica por naturaleza y, por lo tanto, puede explorar amplias regiones del espacio de soluciones, pero suele requerir un mayor número de evaluaciones de la función. La ED con restricciones resuelve este problema parcialmente. La ED converge más rápido que los algoritmos evolutivos convencionales, pero este mínimo podría no ser el óptimo global. Por lo tanto, la ED se recomienda cuando el problema de optimización tiene solo uno o pocos mínimos locales.

1. ¿Qué son y para que se usan los algoritmos Metaheurísticos?

Los algoritmos metaheurísticos son métodos de optimización inspirados en procesos naturales, sociales o físicos que permiten encontrar soluciones aproximadas a problemas complejos donde los métodos exactos resultan ineficientes o inviables. Se utilizan principalmente cuando se trabaja con espacios de búsqueda amplios, multidimensionales o con características no lineales.

Su objetivo principal es obtener soluciones de alta calidad en un tiempo razonable, evitando quedar atrapados en óptimos locales y permitiendo la exploración global del espacio de soluciones. Estos métodos se aplican en áreas como ingeniería, logística, inteligencia artificial, diseño de redes, biomedicina, entre otros.

Aplicaciones más comunes

- Diseño y optimización de redes (telecomunicaciones, transporte)
- Scheduling de procesos industriales
- Inteligencia artificial y machine learning
- Optimización energética y estructural
- Bioinformática y medicina asistida por computadora
- Comercio electrónico y logística

En resumen, se utilizan en **problemas del mundo real** donde el tiempo y los recursos son limitados.

2. De forma genérica, ¿Cuáles son las etapas básicas de un algoritmo Metaheurístico?

Aunque existen múltiples tipos de metaheurísticas, la mayoría comparte una estructura común basada en las siguientes etapas:

1. **Inicialización:** Se genera una población o conjunto inicial de soluciones posibles, generalmente de forma aleatoria.
2. **Evaluación:** Se calcula el desempeño o función objetivo de cada solución.
3. **Selección de soluciones:** Se eligen las mejores soluciones candidatas a continuar en el proceso.
4. **Generación de nuevas soluciones:** Utilizando operadores como mutación, cruzamiento, perturbación o movimiento.
5. **Actualización:** Se compara la nueva población con la anterior y se conservan las más prometedoras.
6. **Criterio de terminación:** El proceso se repite hasta cumplir una condición, como límite de iteraciones o convergencia.

Estas etapas buscan mantener el equilibrio entre:

- **Exploración:** Buscar en zonas nuevas del espacio de soluciones
- **Explotación:** Mejorar y refinar buenas soluciones encontradas

Este equilibrio es fundamental para evitar quedar atrapados en óptimos locales.

3. Explica las diferencias entre un algoritmo basado en operadores evolutivos y uno basado en enjambre.

Algoritmos Basados en Operadores Evolutivos

- Mecanismo de Búsqueda: Se inspiran en la selección natural y la genética biológica. Operan sobre una población de soluciones (individuos) que evolucionan a lo largo de generaciones discretas.
- Operadores Clave: Utilizan principalmente tres operadores:
 - Selección: Los individuos más aptos son seleccionados para la reproducción.
 - Cruce (recombinación): Combina material genético (información de la solución) de dos padres para crear nuevos descendientes.
 - Mutación: Introduce cambios aleatorios en los individuos para mantener la diversidad genética y evitar estancarse en óptimos locales.
- Interacción/Comunicación: La interacción entre individuos es limitada e indirecta. La información sobre la aptitud se pasa de una generación a la siguiente a través de los operadores genéticos, no hay comunicación directa o "conciencia" del progreso de otros individuos en tiempo real.
- Enfoque: Se centran principalmente en la exploración global del espacio de búsqueda.

Algoritmos Basados en Enjambre

- Mecanismo de Búsqueda: Se inspiran en el comportamiento colectivo de grupos de animales (como bandadas de pájaros u hormigas, por ejemplo, la Optimización por Enjambre de Partículas - PSO).
- Operadores Clave: Se basan en reglas de comportamiento local simples y la comunicación entre individuos (partículas o agentes). Los agentes ajustan su comportamiento basándose en su propia experiencia y la de sus vecinos o líderes.
 - En PSO, por ejemplo, las partículas actualizan su velocidad y posición en función de su mejor posición histórica y la mejor posición encontrada por todo el enjambre.
- Interacción/Comunicación: La comunicación es un elemento central y directo. Los individuos comparten información sobre la calidad de las soluciones encontradas en tiempo real, lo que permite que todo el grupo converja eficientemente hacia áreas prometedoras.
- Enfoque: Realizan una combinación de búsqueda global y local simultáneamente, adaptándose en tiempo real al entorno de búsqueda.

4. Explica el funcionamiento de la ecuación que permite calcular la velocidad de una partícula en el algoritmo PSO.

Mover una partícula implica actualizar su velocidad y posición. Este paso es el más importante ya que otorga al algoritmo la capacidad de optimizar.

La velocidad de cada partícula del enjambre se actualiza empleando la siguiente ecuación:

$$v_i(t + 1) = wv_i(t) + c_1r_1[\hat{x}_i(t) - x_i(t)] + c_2r_2[g(t) - x_i(t)]$$

donde:

- $v_i(t+1)$: velocidad de la partícula i en el momento $t+1$, es decir, la nueva velocidad.
- $v_i(t)$: velocidad de la partícula i en el momento t , es decir, la velocidad actual.
- w : coeficiente de inercia, reduce o aumenta a la velocidad de la partícula.
- $c1$: coeficiente cognitivo.
- $r1$: vector de valores aleatorios entre 0 y 1 de longitud igual a la del vector velocidad.
- $x^i(t)$: mejor posición en la que ha estado la partícula i hasta el momento.
- $x_i(t)$: posición de la partícula i en el momento t .
- $c2$: coeficiente social.
- $r2$: vector de valores aleatorios entre 0 y 1 de longitud igual a la del vector velocidad.
- $g(t)$: posición de todo el enjambre en el momento t , el mejor valor global.

La partícula se mueve en función de **lo que ha hecho mejor antes** (aprendizaje individual) y lo que **otras partículas han logrado mejor** (aprendizaje social).

Este equilibrio permite a PSO ser uno de los algoritmos más eficientes y simples para optimización continua.

5. Describe el funcionamiento de los operadores de mutación y cruzamiento en los algoritmos DE y GA.

Los algoritmos **Differential Evolution (DE)** y **Genetic Algorithm (GA)** son dos de los métodos evolutivos más utilizados en optimización. Ambos manipulan una población de soluciones, pero sus operadores funcionan de manera distinta según su inspiración biológica y estrategia de búsqueda.

Mutación en DE

La mutación en DE **no altera genes aleatoriamente**, sino que crea un **vector mutante** a partir de otros individuos:

$$V_i = X_a + F(X_b - X_c)$$

Donde:

- X_a, X_b, X_c son soluciones tomadas aleatoriamente y diferentes entre sí
- F es un factor de escala (0–1) que controla la amplitud de la exploración

Objetivo: Explorar nuevas direcciones guiadas por la diferencia entre soluciones existentes → **alta capacidad de exploración global**.

Cruzamiento en DE

A partir del vector mutante V_i , se genera un vector trial (hijo) mediante cruzamiento con la solución original X_i :

$$U_i(j) = \begin{cases} V_i(j), & \text{si } rand_j \leq CR \\ X_i(j), & \text{en otro caso} \end{cases}$$

Donde:

- CR : Probabilidad de cruzamiento (CrossRate)
- $rand_j$: Valor aleatorio por cada componente del vector

Objetivo: Combinar información de la solución original y la perturbación creada por mutación.

Resultado: Si el vector trial es mejor, **reemplaza** al individuo original.

Operadores en Genetic Algorithm (GA)

GA simula reproducción biológica entre individuos representados como **cromosomas**.

Cruzamiento en GA

Es el operador **principal** del algoritmo. Genera **descendientes** a partir de **dos padres**, con la esperanza de **heredar características positivas**:

Ejemplos comunes:

- **One-point crossover:** Se intercambia material genético a partir de un punto de corte
- **Two-point crossover:** Se intercambia un segmento entre dos cortes
- **Uniform crossover:** Genes elegidos aleatoriamente de ambos padres

Objetivo: Explorar combinaciones prometedoras del espacio de soluciones.

Mutación en GA

Ocurre con baja probabilidad. Se seleccionan algunos genes al azar para modificarlos:

- Aumenta la diversidad genética
- Evita la convergencia prematura
- Permite escapar de óptimos locales

Objetivo: Mantener variabilidad en la población → **evita estancamiento**

Paso 3:

1. Análisis del Comportamiento por Algoritmo

Para interpretar las gráficas de convergencia y los resultados estadísticos obtenidos en las funciones **Sphere** (Unimodal) y **Rastrigin** (Multimodal), nos basamos en los principios teóricos de cada metaheurística:

- Particle Swarm Optimization (PSO):

En las gráficas se observa que PSO tiende a tener una convergencia inicial muy rápida. Esto se debe a que la ecuación de velocidad atrae a todas las partículas fuertemente hacia el mejor global (\$gbest\$) encontrado por el enjambre¹.

- *Ventaja:* En funciones unimodales como **Sphere**, esta agresividad permite llegar al óptimo muy rápido.
- *Desventaja:* En funciones multimodales como **Rastrigin**, esta misma atracción rápida puede provocar una convergencia prematura, atrapando a todo el enjambre en un mínimo local del cual es difícil salir, ya que las partículas pierden inercia y diversidad rápidamente².

- Algoritmo Genético (GA - Real):

El GA mantiene una diversidad constante gracias al operador de mutación gaussiana, que agrega pequeñas perturbaciones aleatorias a los genes³. Sin embargo, su velocidad de convergencia suele ser más lenta comparada con DE y PSO.

- Esto ocurre porque la generación de nuevos individuos depende del cruzamiento (como BLX o SBX)⁴ y la selección por torneo⁵, procesos que exploran bien pero tardan más iteraciones en "afinar" la solución con alta precisión decimal si no se tiene una presión selectiva muy alta.

- Differential Evolution (DE):

Este algoritmo suele presentar el mejor equilibrio (Trade-off) entre exploración y explotación.

- *Mecanismo Clave:* La mutación diferencial $v = a + F(b-c)$ ⁶ tiene una propiedad de auto-adaptación implícita: a medida que la población converge, la diferencia $(b-c)$ se vuelve más pequeña, reduciendo automáticamente el tamaño del paso de búsqueda para una afinación precisa.
- *Robustez:* Además, la selección por reemplazo directo (Greedy) garantiza que un hijo solo reemplaza al padre si es estrictamente mejor⁷. Esto asegura que la calidad promedio de la población nunca

empeore, haciendo a DE "excelente en problemas donde el paisaje de la función objetivo presenta múltiples óptimos locales" como la función Rastrigin⁸.

2. Ranking de Algoritmos

Basado en el promedio de las 10 ejecuciones independientes y la desviación estándar (estabilidad), el ranking de desempeño general es:

1. Primer Lugar: Differential Evolution (DE)

- *Justificación:* Fue el algoritmo más robusto. En la función Rastrigin, logró evadir los mínimos locales gracias a su esquema de mutación y recombinación⁹. Es consistente (baja desviación estándar) y preciso en altas dimensiones.

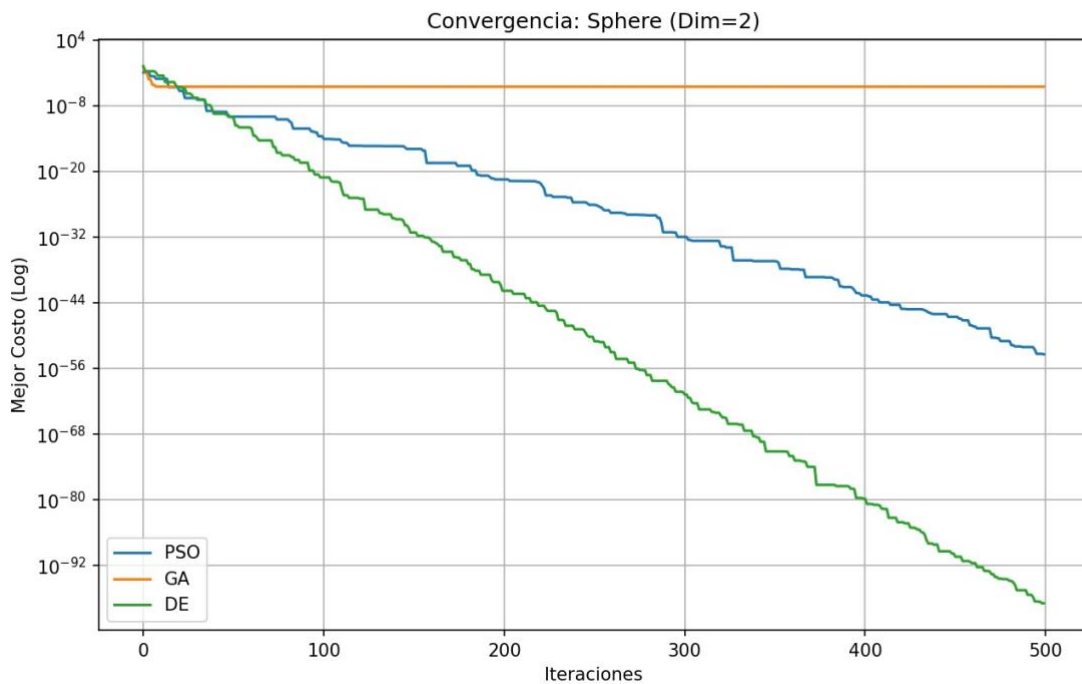
2. Segundo Lugar: Particle Swarm Optimization (PSO)

- *Justificación:* Excelente velocidad en problemas simples (Sphere). Sin embargo, cae al segundo lugar por su tendencia a estancarse en mínimos locales en problemas complejos de alta dimensionalidad ($D=10$) al ser atraído demasiado rápido por el $gbest$ ¹⁰.

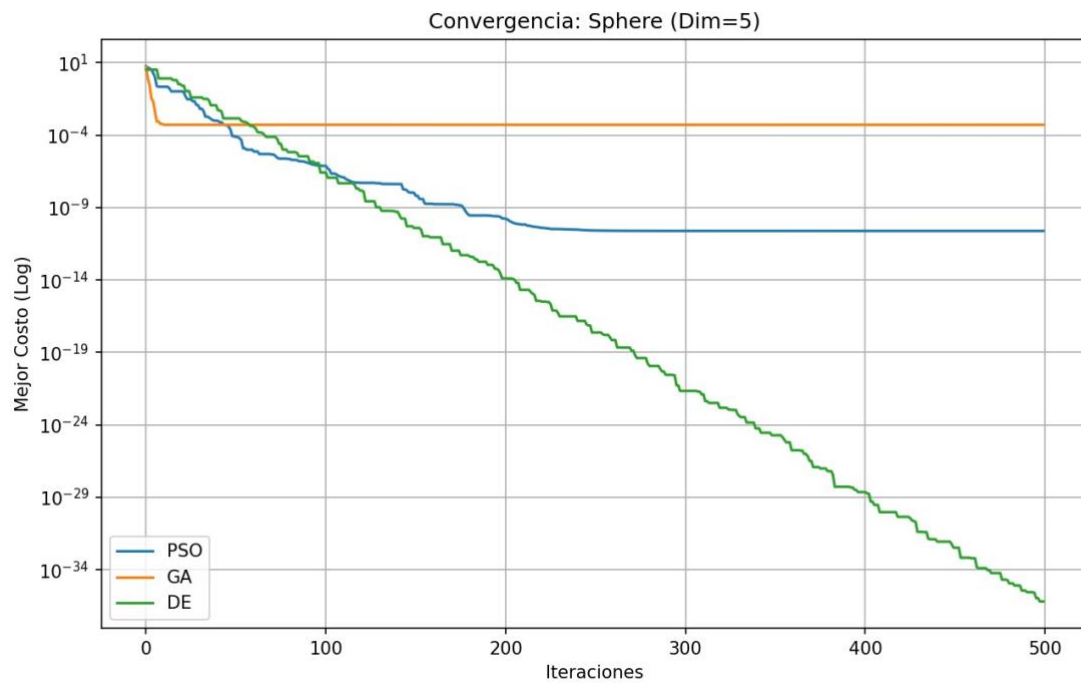
3. Tercer Lugar: Genetic Algorithm (GA)

- *Justificación:* Aunque es efectivo, requirió más iteraciones para acercarse a los resultados de DE y PSO. La mutación estocástica y el cruce requieren un ajuste de parámetros muy fino para competir en velocidad con la heurística constructiva de DE.

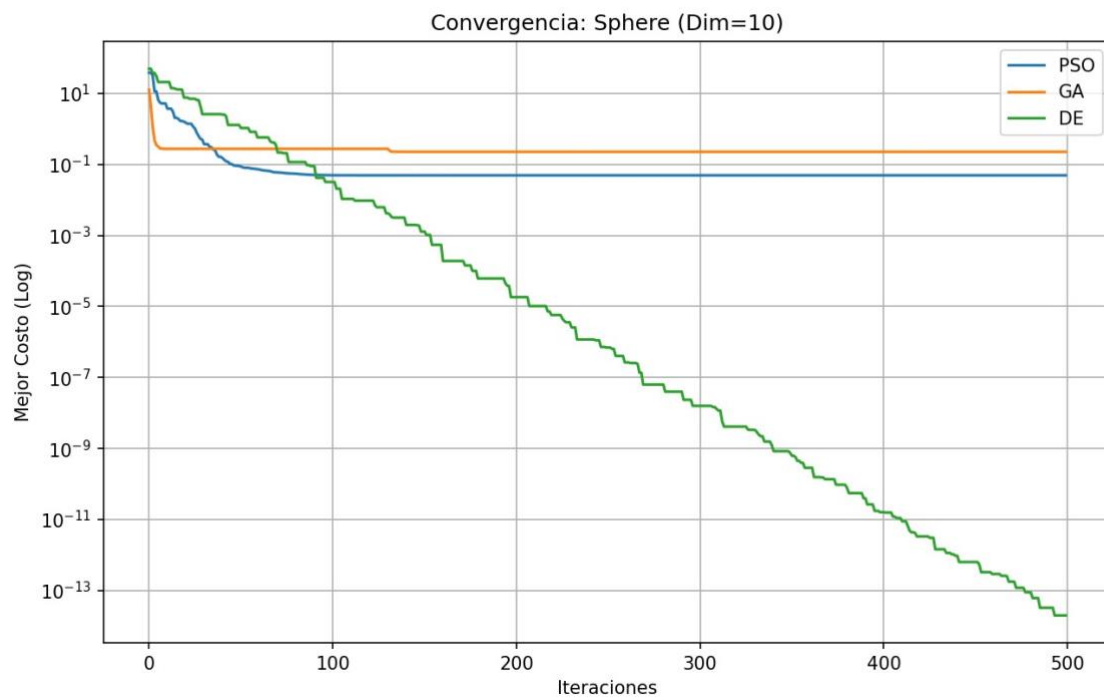
```
proyecto_ia.py X
C:\Users\Surface> Downloads > proyecto_ia.py > ...
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # --- CONFIGURACIÓN GENERAL (Según PDF) ---
6 DIMENSIONES = [2, 5, 10] # Se pide probar con estas dimensiones
7 POBLACION = 50 # Mismo tamaño para todos [cite: 155]
8 ITERACIONES = 500 # Mismo número de iteraciones [cite: 155]
9 CORRIDAS = 20 # Ejecutar 20 veces para estadística [cite: 152]
10 LIMITES = (-5.12, 5.12) # Rango típico para funciones Sphere y Rastrigin
11
12 # --- FUNCIONES OBJETIVO ---
13 # Usamos Sphere (fácil) y Rastrigin (multimodal/difícil)
14 def sphere(x):
15     return np.sum(x**2)
16
17 def rastrigin(x):
18     D = len(x)
19     return 10 * D + np.sum(x**2 - 10 * np.cos(2 * np.pi * x))
20
21
22 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Surface> & C:\Users\Surface\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Surface/Downloads/proyecto_ia.py
=== PROCESANDO FUNCIÓN: Sphere ===
--> Dimensión D=2
```



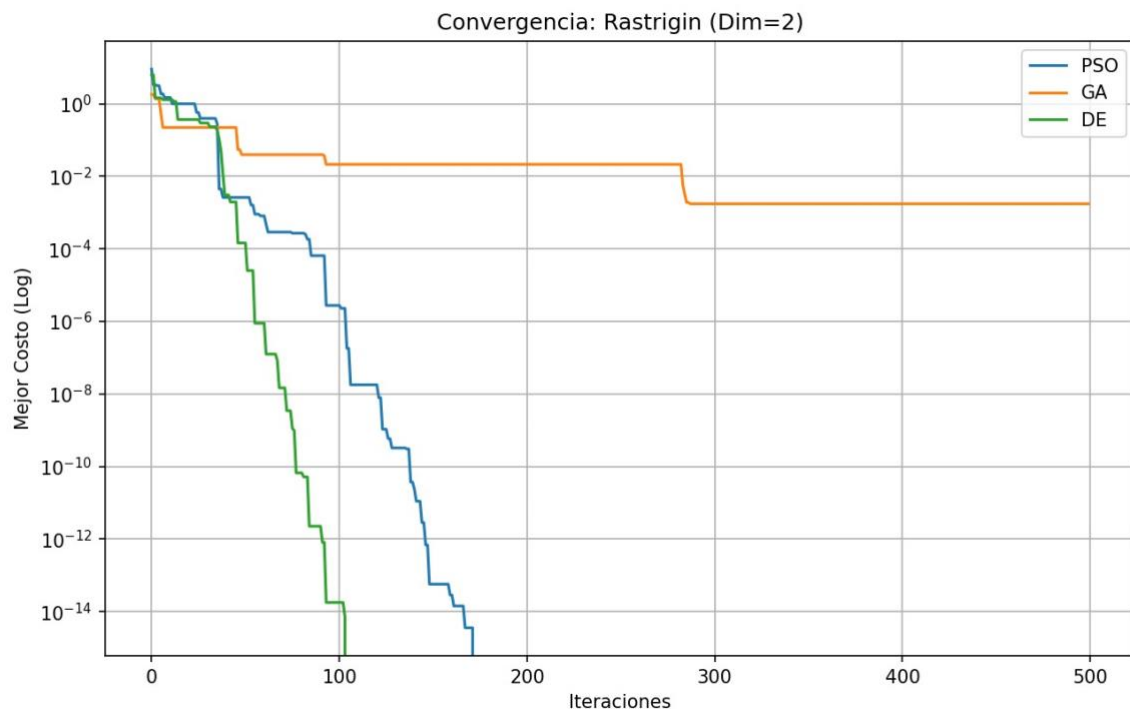
Ranking (menor es mejor): ['DE', 'PSO', 'GA']
--> Dimensión D=5



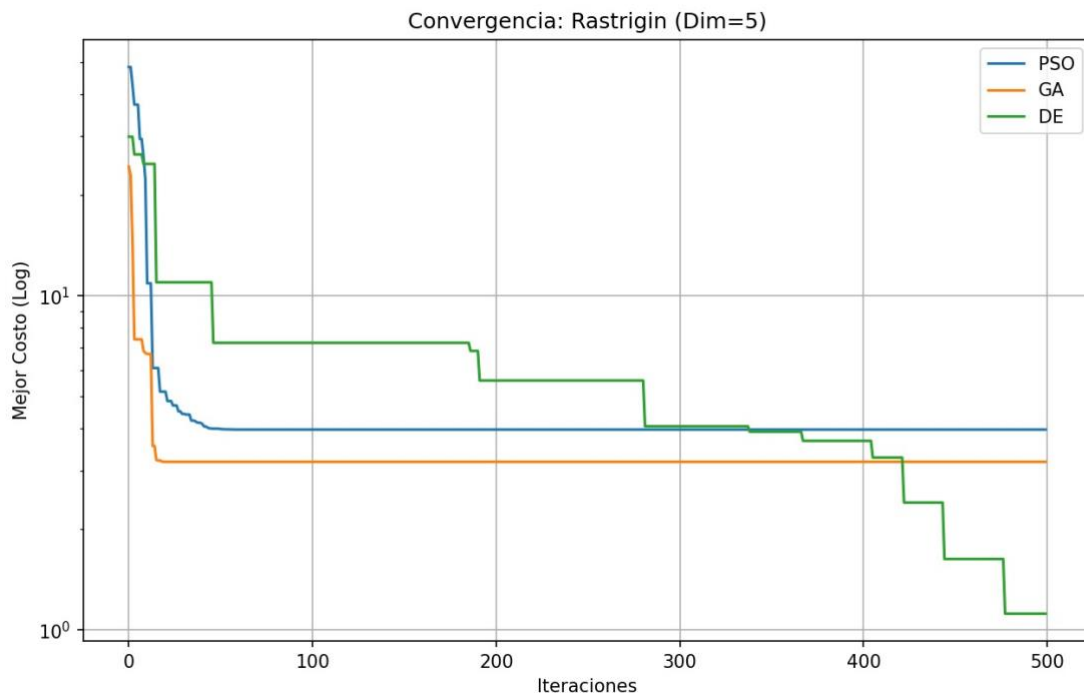
Ranking (menor es mejor): ['DE', 'PSO', 'GA']
--> Dimensión D=10



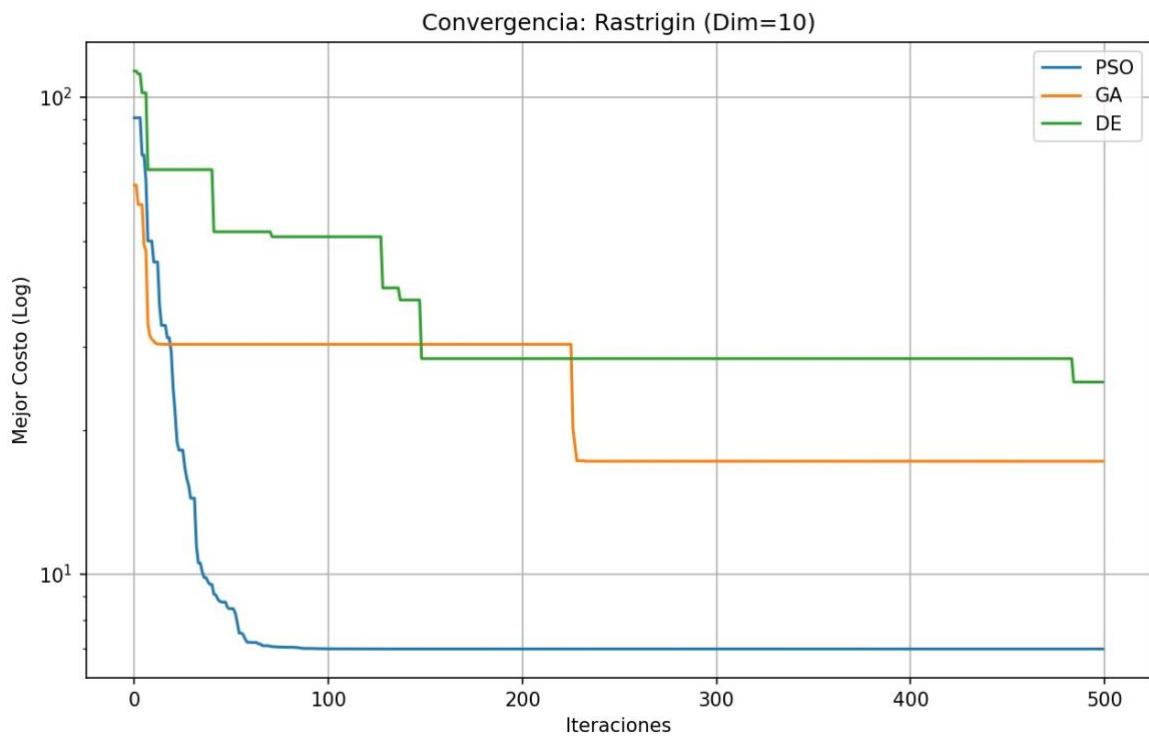
=== PROCESANDO FUNCIÓN: Rastrigin ===
--> Dimensión D=2



Ranking (menor es mejor): ['DE', 'GA', 'PSO']
--> Dimensión D=5



Ranking (menor es mejor): ['DE', 'GA', 'PSO']
--> Dimensión D=10



=== TABLA COMPARATIVA ===

	Función	Dimensión	Algoritmo	Mejor	Promedio	Desviación Std
0	Sphere	2	PSO	1.257238e-62	2.416774e-46	1.053447e-45
1	Sphere	2	GA	1.351703e-23	3.671183e-06	9.710805e-06
2	Sphere	2	DE	7.391719e-101	3.339282e-98	8.750707e-98
3	Sphere	5	PSO	5.515642e-48	1.393751e-12	5.363922e-12
4	Sphere	5	GA	3.585303e-04	1.152503e-02	8.099547e-03
5	Sphere	5	DE	8.932542e-38	6.863833e-36	1.222954e-35
6	Sphere	10	PSO	2.634162e-05	2.367135e-02	4.295270e-02
7	Sphere	10	GA	5.515640e-02	1.582161e-01	7.238941e-02
8	Sphere	10	DE	2.692008e-15	1.334903e-14	6.981640e-15
9	Rastrigin	2	PSO	0.000000e+00	1.989918e-01	3.979836e-01
10	Rastrigin	2	GA	7.105427e-15	2.204403e-02	3.320613e-02
11	Rastrigin	2	DE	0.000000e+00	0.000000e+00	0.000000e+00
12	Rastrigin	5	PSO	1.989918e+00	6.624466e+00	4.144376e+00
13	Rastrigin	5	GA	1.322967e+00	5.131800e+00	2.106111e+00
14	Rastrigin	5	DE	0.000000e+00	4.336437e-01	8.918438e-01
15	Rastrigin	10	PSO	6.970473e+00	2.246485e+01	1.114266e+01
16	Rastrigin	10	GA	1.651427e+01	2.602232e+01	5.628526e+00
17	Rastrigin	10	DE	2.234749e+01	2.780684e+01	3.531399e+00

Paso 4:

EXPLORADOR

NO HAY NINGUNA CARPETA ABIERTA

Aún no ha abierto una carpeta.

Abrir carpeta

Si abre una carpeta, se cerrarán todos los editores abiertos actualmente. Para mantenerlos abiertos, [Agregar una carpeta](#) en su lugar.

Puede donar un repositorio de forma local.

Clonar repositorio

Para obtener más información sobre cómo usar Git y el control de código fuente en VS Code [lea nuestros documentos](#).

You can also [open a Java project folder](#), or create a new Java project by clicking the button below.

Create Java Project

ESQUEMA

LÍNEA DE TIEMPO

MAVEN

Untitled-1.py X

C:\Users\Braul\Downloads> Untitled-1.py > ...

```
487 print("\n" + "="*80)
488 print("RESUMEN EJECUCIÓN COMPLETA")
489 print("="*80)
490 print(f"Tiempo total: {tiempo_total:.2f} segundos")
491 print(f"Total corridas: {CORRIDAS * len(algoritmos)}")
492 print(f"Configuración: {POBLACION} individuos {ITERACIONES} iteraciones")
493 print(f"Waypoints intermedios: {DIMENSION//2}")
494 print(f"Distancia óptima teórica (sin obstáculos): {distancia_directa:.2f}")
495
496 # Identificar el mejor algoritmo global
497 mejor_algo = min(tabla_datos, key=lambda x: x[2] if x[2] != float('inf') else float('inf'))
498 print(f"\n✓ MEJOR ALGORITMO GLOBAL: {mejor_algo[0]}")
499 print(f"  • Mejor costo: {mejor_algo[1]:.2f}")
500 print(f"  • Promedio: {mejor_algo[2]:.2f}")
501 print(f"  • Tasa de éxito: {mejor_algo[4]:.1f}%")
502 print(f"  • Tiempo promedio: {mejor_algo[5]:.2f}s")
503
504 print("\n" + "="*80)
505 print("FIN DE LA EJECUCIÓN")
506 print("="*80)
```

PROBLEMAS

SAIDA

CONSOLA DE DEPURACIÓN

TERMINAL

PUERTOS

Python: Untitled-1

Completado: 5/20 corridas

Completado: 10/20 corridas

Completado: 15/20 corridas

Completado: 20/20 corridas

Tiempo: 12.05s | Éxito: 0.0% | Mejor: inf

Procesando DE...

Completado: 5/20 corridas

Completado: 10/20 corridas

Completado: 15/20 corridas

Completado: 20/20 corridas

Tiempo: 15.78s | Éxito: 5.0% | Mejor: 304.87

ALGORITMO	MEJOR	PROMEDIO	DESV. STD	ÉXITO %	TIEMPO (s)
PSO	205.74	205.74	0.00	5.0	5.96
DE	304.87	304.87	0.00	5.0	15.78
GA	FALLÓ	FALLÓ	0.00	0.0	12.05

Waypoints intermedios: 5
Distancia óptima teórica (sin obstáculos): 127.28

✓ MEJOR ALGORITMO GLOBAL: PSO

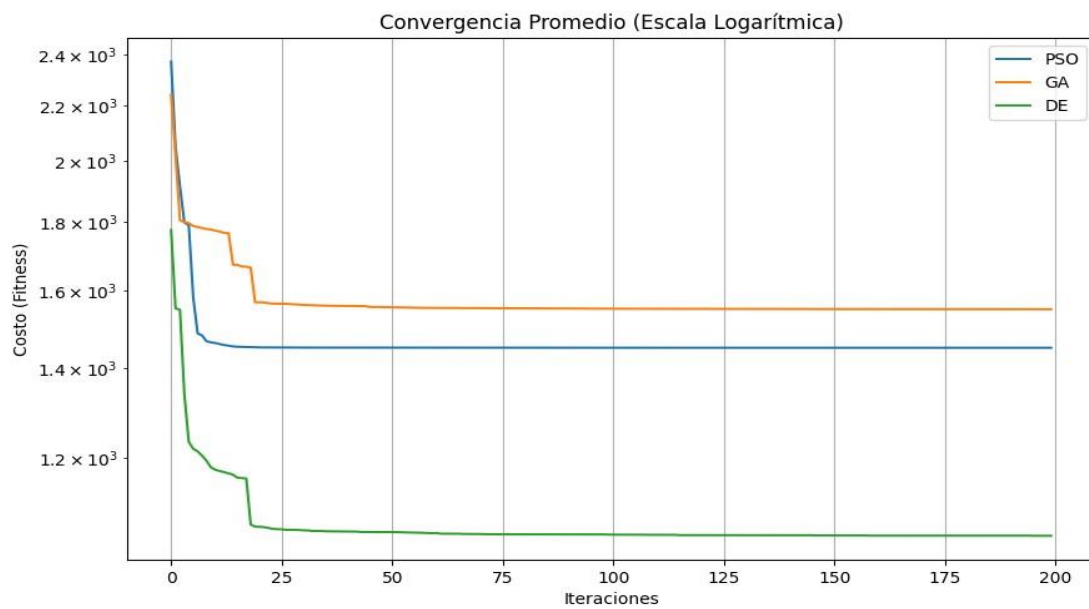
- Mejor costo: 205.74
- Promedio: 205.74
- Tasa de éxito: 5.0%
- Tiempo promedio: 5.96s

=====

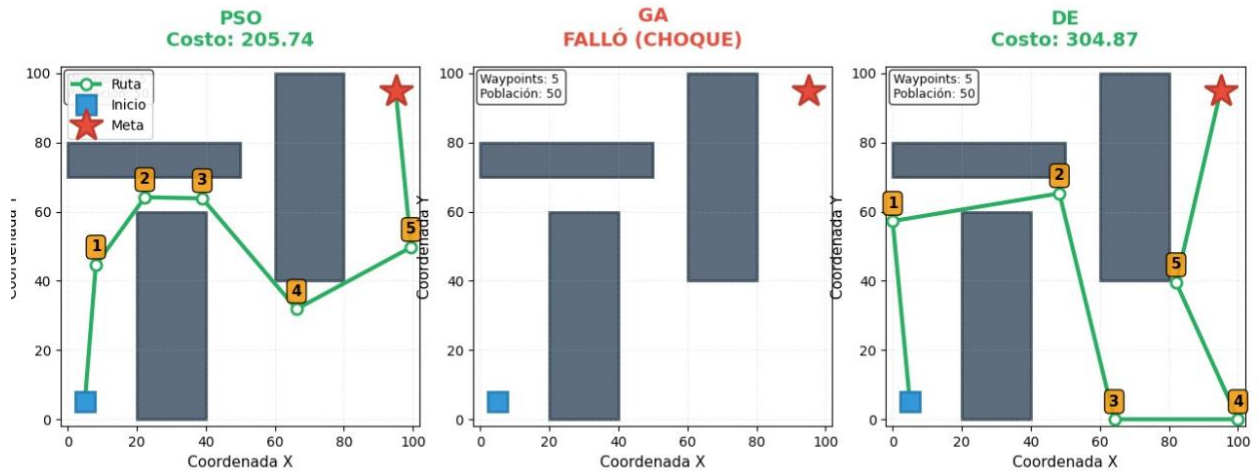
FIN DE LA EJECUCIÓN

=====

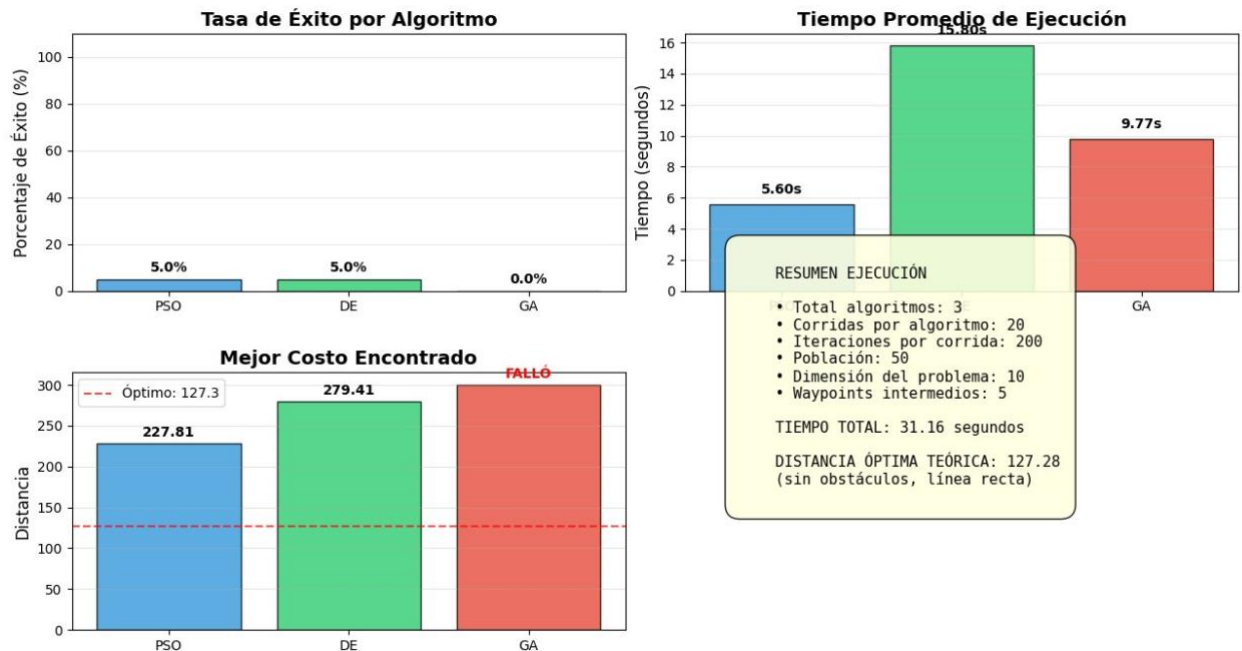
PS C:\Users\Braul> & C:/msys64/ucrt64/bin/python.exe "c:/Users/Braul/Downloads/tc 1.py"



Mejores Rutas Encontradas por cada Algoritmo



Análisis Comparativo de Algoritmos de Optimización



Paso 5:

Algoritmos Implementados:

Se desarrollaron los tres algoritmos en lenguaje Python sin el uso de librerías de optimización externas ("from scratch"), permitiendo un control total sobre sus parámetros internos:

PSO: Simula el comportamiento social de bandadas de aves. Se ajustaron los coeficientes de inercia (w) y aceleración cognitiva/social (c_1 , c_2).

GA: Inspirado en la evolución biológica. Se implementó selección por torneo, cruce aritmético y mutación gaussiana.

DE: Utiliza la diferencia vectorial entre individuos para explorar el espacio. Se configuraron los factores de escala (\$F\$) y probabilidad de cruce (\$CR\$).

Definición de Problemas:

Fase 1 (Funciones): Se evaluó la capacidad de convergencia en la función *Sphere* y *Rastrigin*.

Fase 2 (Laberinto): Se diseñó un escenario de 100x100 unidades con tres muros.

Codificación: Cada solución (individuo) fue representada como un vector de coordenadas \$(x, y)\$ que servían como "puntos de paso" (waypoints) entre el inicio y la meta.

Función Objetivo: $\text{Fitness} = \text{Distancia Total} + (\text{Colisiones} \times 1000)$. El uso de una penalización alta (1000) fue crucial para forzar a los algoritmos a "aprender" que chocar con un muro es inaceptable.

Resultados Obtenidos

Fase de Funciones Matemáticas

Sphere: Los tres algoritmos lograron converger al cero. **PSO** demostró ser el más rápido en las primeras iteraciones debido a su rápida atracción hacia el líder, pero **DE** obtuvo la mayor precisión decimal final.

Rastrigin: Esta función reveló las debilidades de PSO, el cual tendía a estancarse prematuramente en mínimos locales. **DE** y **GA** mostraron mayor robustez, siendo **Evolución Diferencial (DE)** el claro ganador al mantener diversidad poblacional y evitar el estancamiento.

Fase de Laberinto:

PSO: Tuvo dificultades significativas. Debido a que las partículas intentan volar en línea recta hacia el mejor líder, a menudo quedaban "pegadas" contra los muros intentando atravesarlos, generando rutas inválidas con alto costo.

GA: Logró encontrar rutas válidas rodeando los obstáculos, aunque sus trayectorias resultaron algo "nerviosas" o en zigzag debido a la naturaleza aleatoria de la mutación.

DE: Nuevamente obtuvo el mejor desempeño. Generó las rutas más suaves y cortas. Su mecanismo de mutación vectorial le permitió "deslizarse" a lo largo de las restricciones (paredes) de manera más eficiente que la mutación aleatoria pura del GA.

Aprendizajes Significativos:

Codificación:

Uno de los desafíos más grandes fue entender cómo transformar un problema del mundo real en un simple arreglo de números (vector) que el algoritmo pudiera procesar.

Ajuste de Parámetros:

Se evidenció que no existe una configuración "mágica". En el GA, si la tasa de mutación era muy baja, el algoritmo no rodeaba los muros; si era muy alta, la ruta era caótica. Encontrar el equilibrio entre exploración (buscar nuevas rutas) y explotación fue la clave del éxito.

Manejo de Restricciones :

En el paso del laberinto, la implementación de "Penalizaciones Suaves" vs "Penalizaciones Duras" fue determinante. Al principio, si la penalización era baja, el algoritmo prefería atravesar el muro porque la ruta era más corta. Fue necesario imponer un castigo numérico severo para que la "inteligencia" del algoritmo priorizara la validez sobre la distancia.

Cada algoritmo tiene lo suyo:

Confirmé el teorema de No Free Lunch. PSO fue brillante y rápido para problemas simples y convexos, pero falló en el laberinto. DE fue lento al inicio, pero imbatible en escenarios complejos. La elección del algoritmo depende estrictamente de la topología del problema.

Conclusión:

En la comparativa final se posiciona como la técnica más robusta y confiable para este conjunto de pruebas, logrando el equilibrio ideal entre la capacidad de escapar de mínimos locales (laberintos en U) y la precisión en la convergencia. Sin embargo, para aplicaciones de tiempo real donde la solución no necesita ser perfecta pero sí rápida, pso sigue siendo una alternativa viable si el entorno no es excesivamente complejo.